

Linguagens de descrição de hardware

Escola Tecnológica FPFtech

Curso: Técnico em Automação Industrial

Módulo: Eletrônica Digital 2

Semana 02

Prof. Denis Moraes Guimarães

21 de Setembro de 2026

Sumário

- ▶ Linguagens de descrição de hardware;
- ▶ Simulação e síntese;
- ▶ Introdução ao VHDL;
- ▶ Interface, tipos, sinais e concorrência;
- ▶ Modelagem combinacional: porta AND e multiplexador;
- ▶ Modelagem sequencial: flip-flop e contador;
- ▶ Máquinas de estados;
- ▶ Testbench e verificação de resultados;
- ▶ Simulação e formas de onda;
- ▶ Diagnóstico de falhas.

6.1 — O que é HDL?

Hardware Description Language

Linguagem usada para descrever a estrutura e o comportamento de circuitos digitais.

- ▶ Representa portas, conexões, registradores e operações sobre sinais.
- ▶ Permite simular o circuito e verificar seu funcionamento.
- ▶ Um subconjunto da linguagem pode ser sintetizado em hardware.
- ▶ Exemplos de HDL: VHDL, Verilog e SystemVerilog.

Software e descrição de hardware

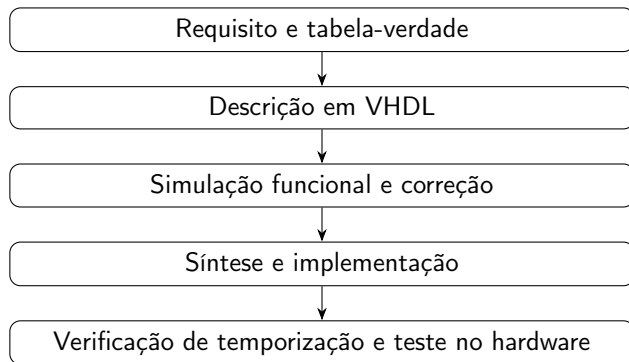
	Software convencional	HDL sintetizável
Descrição	Instruções executadas por um processador	Estrutura e comportamento de um circuito
Execução	Segue o fluxo do programa	Blocos de hardware operam em paralelo
Resultado	Programa executável	Rede de lógica e elementos de memória

Pergunta de projeto

Que circuito esta descrição representa?

HDL também possui comandos sequenciais dentro de processos; isso não significa, por si só, execução por uma CPU.

Do requisito ao circuito



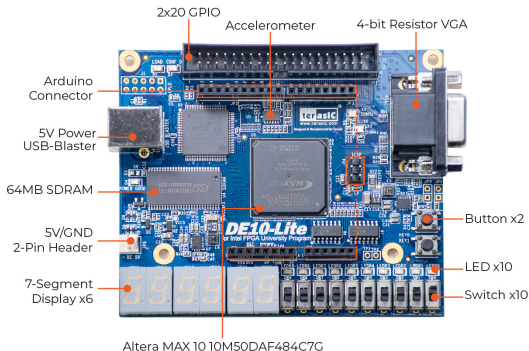
Simulação e síntese

- ▶ **Simulação:** executa um modelo e calcula a evolução dos sinais no tempo.
- ▶ **Síntese:** transforma uma descrição compatível em portas, multiplexadores e registradores.
- ▶ **FPGA:** dispositivo com recursos lógicos configuráveis que podem implementar o circuito.
- ▶ **Testbench:** ambiente de teste; seus estímulos e esperas não fazem parte do circuito a sintetizar.
- ▶ O Quartus Prime sintetiza projetos em VHDL e Verilog. Para estes exemplos, selecione VHDL-2008.

Atenção

Simulação funcional não garante temporização nem requisitos elétricos.

Exemplo de plataforma: placa FPGA



A placa DE10-Lite reúne uma FPGA, chaves, botões, LEDs e displays. Os circuitos descritos em HDL podem controlar essas entradas e saídas.

Conhecendo VHDL

- ▶ VHDL: VHSIC Hardware Description Language.
- ▶ VHSIC: Very High Speed Integrated Circuit.
- ▶ Linguagem fortemente tipada: tipos e conversões precisam ser coerentes.
- ▶ Não diferencia maiúsculas de minúsculas em identificadores comuns.
- ▶ Comentários começam com `--`; instruções normalmente terminam com `;`.
- ▶ Nesta aula, os exemplos e comandos usam **VHDL-2008**.

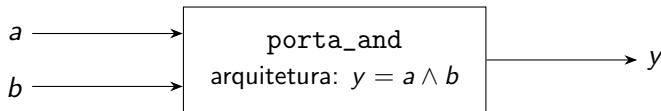
Duas partes essenciais

Entity — interface

Define o nome do circuito e suas portas: entradas e saídas.

Architecture — implementação

Descreve como as saídas são produzidas e declara sinais internos.



Primeiro circuito: porta AND

```
library ieee;  
use ieee.std_logic_1164.all;  
  
entity porta_and is  
  port (a, b : in std_logic;  
        y : out std_logic);  
end entity;  
  
architecture rtl of porta_and is  
begin  
  y <= a and b;  
end architecture;
```

Lendo a descrição da porta AND

- ▶ `library ieee`: torna a biblioteca visível.
- ▶ `std_logic_1164`: disponibiliza tipos lógicos e operações.
- ▶ `in` e `out`: indicam a direção das portas.
- ▶ `rtl`: nome escolhido para a arquitetura.
- ▶ `y <= a and b`: atribuição concorrente de sinal.
- ▶ Quando `a` ou `b` muda, a expressão é reavaliada.

Conexão com a lógica digital

A descrição implementa a mesma função da tabela-verdade de uma porta AND.

Tipos e valores básicos

Tipo	Uso
<code>std_logic</code>	Um sinal lógico: '0', '1' e outros estados.
<code>std_logic_vector(3 downto 0)</code>	Barramento de quatro bits, como "1010".
<code>unsigned</code> , <code>signed</code>	Vetores numéricos do pacote <code>ieee.numeric_std</code> .
<code>integer</code>	Número inteiro; pode ter intervalo limitado.
<code>boolean</code>	Valores <code>true</code> e <code>false</code> .

Estados úteis na simulação

'U': não inicializado; 'X': valor desconhecido; 'Z': alta impedância. São parte dos nove valores de `std_logic`.

Concorrência: circuitos operam em paralelo

```
s1 <= a and b;  
s2 <= c or d;  
y <= s1 xor s2;
```

- ▶ As três atribuições descrevem lógica que existe simultaneamente.
- ▶ A ordem textual dessas atribuições concorrentes não determina uma sequência de execução.
- ▶ Sinais intermediários são declarados antes do begin da arquitetura.

Tempo de simulação

Sem atraso explícito, atualizações se propagam por ciclos delta: etapas de simulação sem avanço do tempo físico.

Sinal e variável

```
signal s : std_logic; -- declaracao na arquitetura
-- Dentro de um process:
-- variable v : std_logic;
-- v := a; -- atualiza a variavel imediatamente
-- s <= a; -- agenda a atualizacao do sinal
```

- ▶ **Sinal:** conecta partes do modelo; atribuição com `<=`.
- ▶ **Variável:** usada em cálculos dentro do processo; atribuição com `:=`.
- ▶ Ler um sinal logo após atribuí-lo no mesmo processo ainda pode retornar seu valor anterior.

6.2 — Formas de modelagem

- ▶ **Fluxo de dados:** expressões e atribuições concorrentes; exemplo: `y <= a and b;`.
- ▶ **Comportamental:** processos, decisões e descrição das operações.
- ▶ **Estrutural:** instanciação e interligação de blocos menores.
- ▶ **RTL:** descreve registradores e a lógica que transforma os dados entre eles.

Escolha de representação

Estilos podem ser combinados. Uma mesma função pode ter descrições diferentes e resultar em hardware equivalente.

Modelagem combinacional: multiplexador 2:1

```
library ieee;
use ieee.std_logic_1164.all;
entity mux2 is
  port (a, b, sel : in std_logic;
        y : out std_logic);
end entity;
architecture rtl of mux2 is
begin
  y <= a when sel = '0' else b;
end architecture;
```

Para seleção binária: sel=0 escolhe a; sel=1 escolhe b. A expressão também escolhe b para outros valores de sel.

O mesmo multiplexador com process

```
architecture comportamental of mux2 is
begin
  process(all)
  begin
    if sel = '0' then
      y <= a;
    else
      y <= b;
    end if;
  end process;
end architecture;
```

process(all) (VHDL-2008) reage aos sinais lidos. Dentro do processo, comandos seguem a ordem textual; o processo opera concorrentemente com os demais blocos.

Cuidado: atribuição incompleta infere memória

```
process(all)
begin
  if habilita = '1' then
    y <= a;
  end if;
end process;
```

- ▶ Quando habilita não é 1, a saída mantém seu valor anterior.
- ▶ Na síntese, esse comportamento pode inferir um latch.
- ▶ Para lógica combinacional, atribua todas as saídas em todos os caminhos: use else ou valores padrão.

Modelagem sequencial: flip-flop D

```
library ieee;
use ieee.std_logic_1164.all;
entity ff_d is
  port (clk, rst, d : in std_logic;
        q : out std_logic);
end entity;
architecture rtl of ff_d is
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then q <= '0';
      else q <= d;
      end if;
    end if;
  end process;
end architecture;
```

Interpretando o flip-flop

- ▶ `rising_edge(clk)` detecta a borda de subida do clock.
- ▶ O reset é **síncrono**: só atua na borda de subida.
- ▶ Na borda: se `rst=1`, `q` recebe zero; caso contrário, recebe `d`.
- ▶ Entre bordas, `q` preserva o valor armazenado.
- ▶ Sem reset ou inicialização, `q` pode começar como U na simulação.

Previsão

Se `d` mudar de 0 para 1 entre duas bordas, `q` só capturará 1 na próxima borda de subida, com `rst=0`.

Contador binário de 4 bits

Comportamento

Conta de 0 a 15 nas bordas de subida do clock e retorna a 0 após 15.

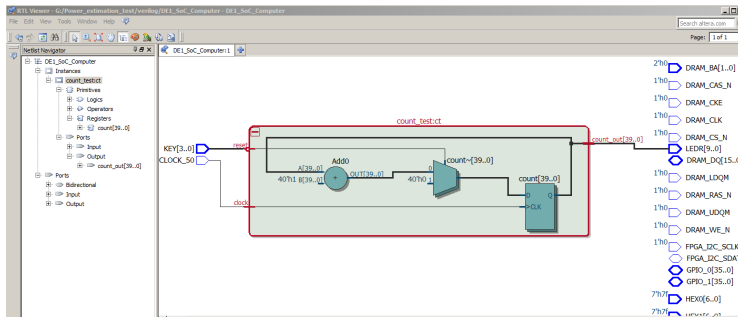
- ▶ `rst=1`: zera o contador na próxima borda de subida.
- ▶ `rst=0` e `en=1`: incrementa o valor armazenado.
- ▶ `rst=0` e `en=0`: mantém a contagem.
- ▶ `unsigned(3 downto 0)` representa um número sem sinal.

Borda	rst	en	q após a borda
1	1	0	0000
2	0	1	0001
3	0	1	0010
4	0	0	0010
5	0	1	0011

Contador: implementação em VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity contador4 is
  port (clk, rst, en : in std_logic;
        q : out std_logic_vector(3 downto 0));
end entity;
architecture rtl of contador4 is
  signal cont : unsigned(3 downto 0);
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then cont <= (others => '0');
      elsif en = '1' then cont <= cont + 1;
      end if;
    end if;
  end process;
  q <= std_logic_vector(cont);
end architecture;
```

Contador no Quartus: visualização RTL



Observe: somador, multiplexador, registrador e realimentação.

A captura mostra um exemplo de contador de 40 bits publicado pela Cornell. Ajuda a reconhecer os blocos de um contador; o modelo desta aula possui 4 bits e habilitação.

Máquina de estados: controle de uma esteira

FSM — Finite State Machine

Circuito sequencial cujo comportamento depende do estado atual e das entradas.

- ▶ **PARADO:** motor desligado, aguardando `iniciar=1`.
- ▶ **MOVENDO:** motor ligado até `sensor=1`.
- ▶ **CONCLUIDO:** motor desligado e `pronto=1`.
- ▶ Permanece concluído enquanto `iniciar=1`; retorna a parado quando `iniciar=0`.

Modelo didático: entradas já sincronizadas com o clock. Reset síncrono leva a PARADO. As saídas dependem apenas do estado: máquina de Moore.

Aplicação: transporte de peças na indústria

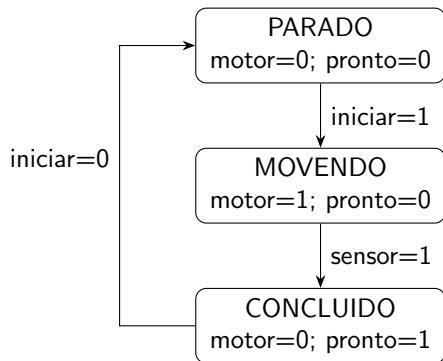


Relação com o modelo

- ▶ Iniciar: solicita o movimento.
- ▶ Motor: aciona o transporte.
- ▶ Sensor: indica a chegada da peça.
- ▶ Pronto: sinaliza o fim do ciclo.

A foto contextualiza o transporte industrial. A máquina de estados apresentada é um modelo didático simplificado.

Esteira: diagrama de estados



As transições ocorrem na borda de subida. Sem a condição indicada, mantém-se o estado. Reset tem prioridade e leva qualquer estado a PARADO.

Esteira em VHDL: interface e estados

```
library ieee;
use ieee.std_logic_1164.all;
entity controle_esteira is
  port (clk, rst, iniciar, sensor : in std_logic;
        motor, pronto : out std_logic);
end entity;
architecture rtl of controle_esteira is
  type estado_t is (PARADO, MOVENDO, CONCLUIDO);
  signal estado : estado_t;
begin
```

- ▶ O tipo enumerado dá nomes aos estados do controle.
- ▶ O sinal estado armazena o estado atual.
- ▶ A síntese determina uma codificação binária para os estados.

O processo e as saídas a seguir completam esta arquitetura.

Esteira em VHDL: transições

```
process(clk)
begin
  if rising_edge(clk) then
    if rst = '1' then
      estado <= PARADO;
    else
      case estado is
        when PARADO =>
          if iniciar = '1' then estado <= MOVENDO;
          end if;
        when MOVENDO =>
          if sensor = '1' then estado <= CONCLUIDO;
          end if;
        when CONCLUIDO =>
          if iniciar = '0' then estado <= PARADO;
          end if;
        end case;
      end if;
    end if;
  end process;
```

Esteira em VHDL: saídas e sequência

```

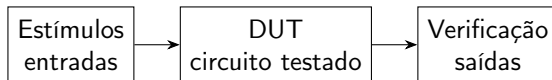
motor <= '1' when estado = MOVENDO else '0';
pronto <= '1' when estado = CONCLUIDO else '0';
end architecture;

```

Borda	Entradas antes da borda			Após a borda		
	rst	iniciar	sensor	Estado	motor	pronto
1	1	0	0	PARADO	0	0
2	0	1	0	MOVENDO	1	0
3	0	1	0	MOVENDO	1	0
4	0	1	1	CONCLUIDO	0	1
5	0	1	0	CONCLUIDO	0	1
6	0	0	0	PARADO	0	0

As saídas refletem o novo estado após as atualizações dos sinais na simulação.

Simulação básica: o papel do testbench



- ▶ DUT: Design Under Test, o circuito sob teste.
- ▶ O testbench normalmente possui uma entity sem portas.
- ▶ Instancia o DUT, aplica entradas e compara saídas com o esperado.
- ▶ Esperas representam tempo simulado, não tempo real de execução.

Testbench da AND: interface e instanciação

```
library ieee;
use ieee.std_logic_1164.all;
entity tb_porta_and is
end entity;
architecture sim of tb_porta_and is
    signal a, b : std_logic := '0';
    signal y : std_logic;
begin
    dut: entity work.porta_and(rtl)
        port map (a => a, b => b, y => y);
```

- ▶ work: biblioteca de trabalho do projeto.
- ▶ port map: associa portas do DUT aos sinais do testbench.
- ▶ O processo de estímulos vem a seguir, na mesma arquitetura.

Testbench da AND: estímulos e verificações

```
estimulos: process
begin
  a <= '0'; b <= '0'; wait for 10 ns;
  assert y = '0' report "Falha: 00" severity error;
  a <= '0'; b <= '1'; wait for 10 ns;
  assert y = '0' report "Falha: 01" severity error;
  a <= '1'; b <= '0'; wait for 10 ns;
  assert y = '0' report "Falha: 10" severity error;
  a <= '1'; b <= '1'; wait for 10 ns;
  assert y = '1' report "Falha: 11" severity error;
  report "Teste concluido: 4 casos" severity note;
  std.env.finish;
  wait;
end process;
end architecture;
```

Cada assert exige a condição verdadeira. A espera permite que a saída estabilize antes da comparação.

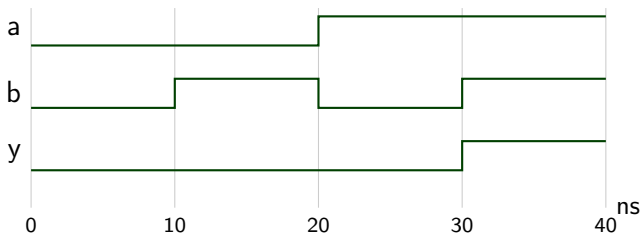
Executando com GHDL e GTKWave

No terminal, dentro da pasta exemplos, com as ferramentas instaladas:

```
ghdl -a --std=08 porta_and.vhd tb_porta_and.vhd  
ghdl -e --std=08 tb_porta_and  
ghdl -r --std=08 tb_porta_and --assert-level=error --vcd=and.vcd  
gtkwave and.vcd
```

- ▶ -a: analisa os arquivos; -e: elabora o topo.
- ▶ -r: executa; -vcd: grava formas de onda.
- ▶ -assert-level=error: interrompe ao detectar erro.
- ▶ Resultado esperado: quatro casos verificados e término em 40 ns.

Formas de onda esperadas da AND



Intervalo (ns)	a	b	y
0–10	0	0	0
10–20	0	1	0
20–30	1	0	0
30–40	1	1	1

Diagrama idealizado após estabilização; omite ciclos delta e o estado inicial U de *y*.

Como investigar uma falha?

- ▶ Erro de análise: confira ponto e vírgula, nomes, tipos e bibliotecas.
- ▶ Erro de elaboração: confira o nome do topo e as entidades instanciadas.
- ▶ Saída U: procure sinais não inicializados ou sem atribuição.
- ▶ Saída X: investigue valores desconhecidos ou drivers em conflito.
- ▶ Falha em assert: compare estímulo, instante de leitura e resultado esperado.
- ▶ Em circuitos sequenciais, verifique borda de clock e comportamento do reset.